

- Getting Started
- Chai aur HTML
- Chai aur Git
 - Welcome
 - Git and GitHub
 - Terminology
 - Behind the scenes
 - Branches in Git
 - Diff, Stash, Tags
 - Managing History
 - Collaborate with Github
- Chai aur C++
- Chai aur Django
- Chai aur SQL
- Chai aur DevOps

Git and GitHub

Let's start with the basics. Git is a version control system that allows you to track changes to your files and collaborate with others. It is used to manage the history of your code and to merge changes from different branches. I can understand that as of now these terms like version control, branches, and merges are not familiar to you. But don't worry, we will learn them in this tutorial.

Git and Github are different

Git is a version control system that is used to track changes to your files. It is a free and open-source software that is available for Windows, macOS, and Linux. Remember, GIT is a software and can be installed on your computer.

Github is a web-based hosting service for Git repositories. Github is an online platform that allows you to store and share your code with others. It is a popular platform for developers to collaborate on projects and to share code. It is not that Github is the only provider of Git repositories, but it is one of the most popular ones.

A little on version control systems

Version control systems are used to manage the history of your code. They allow you to track changes to your files and to collaborate with others. Version control systems are essential for software development. Consider version control as a checkpoint in game. You can move to any time in the game and you can always go back to the previous checkpoint. This is the same concept in software development.

Before Git became mainstream, version control systems were used by developers to manage their code. They were called SCCS (Source Code Control System). SCCS was a proprietary software that was used to manage the history of code. It was expensive and not very user-friendly. Git was created to replace SCCS and to make version control more accessible and user-friendly. Some common version control systems are Subversion (SVN), CVS, and Perforce.

Learning Path

In this tutorial, we will learn the basics of Git and Github. We will start with the basics and then move on to more advanced topics. We will also learn how to use Git and Github for collaboration and version control. By the end of this tutorial, you will have a good understanding of Git and Github and will be able to use them to manage your code effectively.

We will go in this journey something like this:

- Get the basics
- Use it daily
- Face the problems
- Solve them
- Learn more

We will focus more on Git first, once you understand git, moving towards Github will be easy.

Install Git

To install Git, you can use command line or you can visit official website and download the installer for your operating system. Git is available for Windows, macOS, and Linux and is available at <https://git-scm.com/downloads>.

Account on Github

Another step that you have to follow is to create an account on Github. I will later walk you through the process of linking your Github account with your Machine. You cannot push your code to Github without ssh-key setup. Password authentication is not recommended and these days it is not possible to use it. So, you need to setup ssh-key authentication. We will cover that in a later part of the tutorial.

Conclusion

In this part, we have learned the basics of Git and Github. We have also learned how to install Git and Github. We have also learned about the importance of version control systems and how they are used in software development.

Start your journey with ChaiCode

All of our courses are available on chaicode.com. Feel free to check them out.

On this page

Overview

- Git and Github are different
- A little on version control systems
- Learning Path
- Install Git
- Account on Github
- Conclusion

Learning Path

- Install Git
- Account on Github
- Conclusion

Learning Path

- Install Git
- Account on Github
- Conclusion

Learning Path

- Install Git
- Account on Github
- Conclusion

Learning Path

- Install Git
- Account on Github
- Conclusion

- Getting Started
- Chai aur HTML
- Chai aur Git
 - Welcome
 - Git and GitHub
 - Terminology
 - Behind the scenes
 - Branches in Git
 - Diff, Stash, Tags
 - Managing History
 - Collaborate with Github
- Chai aur C++
- Chai aur Django
- Chai aur SQL
- Chai aur DevOps

Git behind the scenes

Git is a version control system that allows you to track changes to your files and folders. It is a powerful tool that can help you manage your code more effectively. In this section, we will explore the basics of how git works internally.

Git Snapshots

A git snapshot is a point in time in the history of your code. It represents a specific version of your code, including all the files and folders that were present at that time. Each snapshot is identified by a unique hash code, which is a string of characters that represents the contents of the snapshot.

A snapshot is not an image, it's just a representation of the code at a specific point in time. Snapshot is a loose term that is used when git stores information about the code in a locally stored key-value based database. Everything is stored as an object and each object is identified by a unique hash code.

3 Musketeers of Git

The three musketeers of git are:

- Commit Object
- Tree Object
- Blob Object

Commit Object

Each commit in the project is stored in `.git` folder in the form of a commit object. A commit object contains the following information:

- Tree Object
- Parent Commit Object
- Author
- Committer
- Commit Message

Tree Object

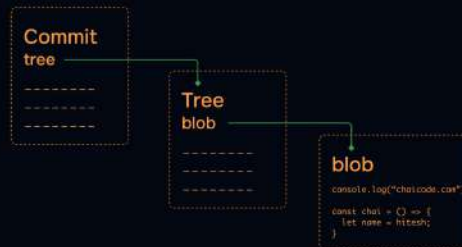
Tree Object is a container for all the files and folders in the project. It contains the following information:

- File Mode
- File Name
- File Hash
- Parent Tree Object

Everything is stored as key-value pairs in the tree object. The key is the file name and the value is the file hash.

Blob Object

Blob Object is present in the tree object and contains the actual file content. This is the place where the file content is stored.



Helpful commands

Here are some helpful commands that you can use to explore the git internals:

```
***  
git show -s --pretty=raw <commit-hash>
```

Grab tree id from the above command and use it in the following command to get the tree object:

```
***  
git ls-tree <tree-id>
```

Grab tree id from the above command and use it in the following command to get the blob object:

```
***  
git show <blob-id>
```

Grab tree id from the above command and use it in the following command to get the commit object:

```
***  
git cat-file -p <commit-id>
```

Start your journey with ChaiCode

All of our courses are available on chaicode.com. Feel free to check them out.

♥ Compiled by: Hitesh Choudhary

Last updated: Apr 14, 2025

← Previous

Next →

Terminology

Branches in Git

On this page

Overview

- Git Snapshots
- 3 Musketeers of Git
- Commit Object
- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Commit Object

- Tree Object
- Blob Object
- Helpful commands

Getting Started

- Chai for HTML
- Chai for Git
- Chai for C++
- Chai for Django
- Chai for GDB
- Chai for Docker

Branches in Git

Branches are a way to work on different versions of a project at the same time. They allow you to create a separate line of development that can be worked on independently of the main branch. This can be useful when you want to make changes to a project without affecting the main branch or when you want to work on a new feature or bug fix.



Some developers can work on `feature`, some can work on `feature`, some can work on `feature`, and some can work on `feature`. This is a good example of how branches can be used in git.

HEAD in git

The HEAD is a pointer to the current branch that you are working on. It points to the latest commit in the current branch. When you create a new branch, it is automatically set as the HEAD of that branch.

The default branch used to be `master`, but it is now called `main`. There is nothing special about `main`, it is just a convention.

Creating a new branch

To create a new branch, you can use the following command:

```
git branch
git branch bug-fix
git switch bug-fix
git log
git checkout -b bug-fix
git switch -d bug-fix
git checkout main
```

Some points to note:

- `git branch` - This command lists all the branches in the current repository.
- `git branch bug-fix` - This command creates a new branch called `bug-fix`.
- `git checkout bug-fix` - This command switches to the `bug-fix` branch.
- `git log` - This command shows the commit history for the current branch.
- `git checkout -b bug-fix` - This command switches to the `bug-fix` branch and creates a new branch called `bug-fix`.
- `git switch -d bug-fix` - This command switches to the `bug-fix` branch and creates a new branch called `bug-fix`.

Commit before switching to a branch
Go to `git` folder and checkout to the HEAD file

Merging branches

- Merging is about bringing changes from one branch to another.
- to Git, there are two types of merges:
 - Fast Forward Merge (if branches have not diverged)
 - 3-Way Merge (if branches have diverged)

fast forward merge

This is a type of merge that you are trying to merge in usually ahead and there are no conflicts.

When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

```
git checkout main
git merge bug-fix
```



Some points to note:

- `git checkout main` - This command switches to the `main` branch.
- `git merge bug-fix` - This command merges the `bug-fix` branch into the `main` branch.

This is a fast forward merge. It means that the commit in the `bug-fix` branch is directly merged into the `main` branch. This can be useful when you want to merge a branch that has already been pushed to the remote repository.

3 Way merge



In this type of merge, the main branch has additional commits that are not present in the `bug-fix` branch. This is not a fast forward merge, then git looks at 3 different commit (ancestor) of branches + tip of each branch and combines the changes into one merge commit.

When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

```
git checkout main
git merge bug-fix
```

If the command are same, what is the difference between fast forward and not fast forward merge?

The difference is resolving the conflicts. In a fast forward merge, there are no conflicts. But in a not fast forward merge, there are conflicts, and there are no shortcuts to resolve them. You have to manually resolve the conflicts. Decide what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts.



Managing conflicts

There is no magic button to resolve conflicts. You have to manually resolve the conflicts. Decide what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts. I personally use VSCode merge tool. Github also has a merge tool that can help you resolve the conflicts. But most of the time I handle them in VSCode and it gives me all the options to resolve the conflicts.

Overall it sounds scary to beginners but it is not, it's all about communication and understanding the code situation with your team members.

Rename a branch

You can rename a branch using the following command:

```
git branch -m old-branch-name new-branch-name
```

Delete a branch

You can delete a branch using the following command:

```
git branch -d branch-name
```

Checkout a branch

You can checkout a branch using the following command:

```
git checkout branch-name
```

Checkout a branch means that you are going to work on that branch. You can checkout any branch you want.

List all branches

You can list all branches using the following command:

```
git branch
```

List all branches means that you are going to see all the branches in your repository.

Conclusion

In this section, we have learned about the different types of merges and how to resolve conflicts. We have also learned about the importance of branching and merging in Git and Github.

Start your journey with ChaiCode

All of our courses are available on chaidocs.com, feel free to check them out.

Completed by: [ChaiCode](https://chaidocs.com)

Last updated: Apr 14, 2025

Behind the scenes

Diff, Stash, Tags

- Getting Started
- Chai aur HTML
- Chai aur Git
 - Welcome
 - Git and GitHub
 - Terminology
 - Behind the scenes
 - Branches in Git
 - Diff, Stash, Tags
 - Managing History
 - Collaborate with Github
- Chai aur C++
- Chai aur Django
- Chai aur SQL
- Chai aur DevOps

Managing History

This guide will help you understand how to manage your Git history effectively.

Merge commits

A merge commit is a commit that combines two or more commits into one. It is created when you merge two or more branches into a single branch. The merge commit contains all the changes from the original branches, and it is used to keep the project history clean and easy to understand.



Rebase in git

Git rebase is a powerful Git feature used to change the base of a branch. It effectively allows you to move a branch to a new starting point, usually a different commit, by "replaying" the commits from the original base onto the new base. This can be useful for keeping a cleaner, linear project history.

Some people like to use rebase over the merge command because it allows you to keep the commit history cleaner and easier to understand. It also allows you to make changes to the code without affecting the original branch.

Here's a flow example of using git rebase with all the commands involved:

Suppose you have a feature branch called feature-branch that you want to rebase onto the main branch.



Ensure you are on the branch you want to rebase

```
***
git checkout feature-branch
git rebase main
```

This will replay the commits from feature-branch on top of the latest changes in main.

Resolve any conflicts

If there are any conflicts, you will need to resolve them manually. You can use the merge tool in VSCode to resolve the conflicts.

```
***
git add <resolved-files>
git rebase --continue
```

Try to avoid `--force` option when using rebase. It can cause issues with the project history. I have seen many horror stories of people using `--force` to fix conflicts.

Git reflog

Git reflog is a command that shows you the history of your commits. It allows you to see the changes that you have made to your repository over time. This can be useful for debugging and understanding the history of your project.

View the reflog:

```
***
git reflog
```

This will show you the history of your commits. You can use the number at the end of each line to access the commit that you want to view.

Find specific commit

You can find a specific commit using the following command:

```
***
git reflog <commit-hash>
```

Recover lost commits or changes

If you accidentally deleted a branch or made changes that are no longer visible in the commit history, you can often recover them using the reflog. First, find the reference to the commit where the branch or changes existed, and then reset your branch to that reference.

```
***
git reflog <commit-hash>
git reset --hard <commit-hash>
```

or you can use `HEAD@{n}` to reset to the nth commit before the one you want to reset to.

```
***
git reflog <commit-hash>
git reset --hard HEAD@{1}
```

Conclusion

In this guide, we've covered important aspects of managing Git history through rebase and reflog. We've learned how rebase can help maintain a cleaner, more linear project history, and how reflog can help recover lost commits or changes.

Start your journey with ChaiCode

All of our courses are available on chalicecode.com. Feel free to check them out.

♥ Compiled by: Hitesh Choudhary

Last updated: Apr 14, 2025

← Previous

Next →

Diff, Stash, Tags

Collaborate with Github

On this page

- Overview
- Merge commits
- Rebase in git
- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

- Ensure you are on the branch you want to rebase
- Resolve any conflicts
- Git reflog
- View the reflog
- Find specific commit
- Recover lost commits or changes
- Conclusion

About Github

This guide will help you get started with Github.

What is Github?

Github is a web-based distributed version control system. It is a popular platform for developers to collaborate on projects and share code. Github provides a user-friendly interface for managing and tracking changes to your code, as well as a platform for hosting and sharing your projects with others.

Some other alternatives of Github are:

- Bitbucket
- SourceTree
- GitLab

But repository popular than these days is Github.

Github Account

Creating a Github account is a free and easy. You can create an account by visiting the [Github website](#) and clicking on the "Sign up" button. You will be prompted to enter your email address and password, and then you will be redirected to the Github homepage.

Once you have created an account, you can start using Github to host and collaborate on your projects. Github provides a range of features and tools that make it easy to manage and track your code, including issues, pull requests, and code reviews.

Configuring Git

Before we start, it's already, you need to configure your git config file. You can do this by running the following command:

```
git config --global user.name "Your Name"
git config --global user.email "Your Email"
```

This will set your email and name as your global settings. You can change these settings at any time by running the same command again.

Now you can check your config settings:

```
git config --list
```

You will show you all the settings that you have changed.

Setup SSH Key

Before we start, it's already, you need to setup an SSH key and add it to your Github account. You can do this by following the instructions on the [Github website](#).

You can find the exact steps on the website for both Windows and Mac OS. The steps are same for both, only slight don't need to add the key to their keychain.

1. Generate a new SSH key

To generate a new SSH key, open the terminal and run the following command:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Here ed25519 is the type of key that you are generating. This creates a new SSH key, using the provided email as label.

2. Save the key

After generating the key, you need to save it to your computer. You can do this by running the following command:

```
cat ~/.ssh/id_ed25519.pub
```

At this point, you can copy the output for the key or you can have it saved. If you have it saved, the key will be saved without a passphrase.

3. Add key to your ssh-agent

After saving the key, you need to add it to your ssh-agent. You can do this by running the following command:

```
ssh-add ~/.ssh/id_ed25519
```

Here it's already, when above info is saved information, so Github has a lot of information on this. There is no need to repeating it here.

4. Add key to Github

Now you can add the key to your Github account. You can do this by following the instructions on the [Github website](#).

Publish Code to Remote Repository

Now that you have setup your git key and added it to your Github account, you can start pushing your code to the remote repository.

Create a new file on your system first, add some code and commit it.

```
git init
git add .
git commit -m "Initial commit"
```

Remote URL, Setting

You can check the remote url setting by running the following command:

```
git remote -v
```

You will show you the remote url of your repository.

Add Remote Repository

You can add a remote repository by running the following command:

```
git remote add origin
```

Here origin is the name of the remote repository that you want to add and origin is the name of the remote repository. This origin is used to refer to the remote repository in the future.

```
git remote add origin https://github.com/yourusername/yourrepository.git
```

Pushing Code

git push remote-name branch-name

Here origin is the name of the remote repository that you want to push to and branch is the name of the branch that you want to push.

```
git push origin main
```

Setup an upstream remote

Setting up an upstream remote is useful when you want to keep your local repository up to date with the remote repository. It allows you to track and merge changes from the remote repository into your local repository.

To set up an upstream remote, you can use the following command:

```
git remote add upstream https://github.com/yourusername/yourrepository.git
```

or you can use shorthand:

```
git remote add upstream -f
```

You can check the list of pushing your code to the remote repository.

```
git push upstream main
```

This will set up an upstream remote and push your code to the remote repository.

This will allow you to run future commands like `git pull` and `git push` without specifying the remote name.

Get code from remote repository

Now we have setup to get code from remote repository.

- fetch the code
- pull the code

Fetch the code means that you are going to download the code from the remote repository to your local repository. Pull the code means that you are going to download the code from the remote repository and merge it with your local repository.



Fetch code

To fetch code from a remote repository, you can use the following command:

```
git fetch remote-name
```

Here origin is the name of the remote repository that you want to fetch from.

Pull code

To pull code from a remote repository you can use the following command:

```
git pull remote-name branch-name
```

Here origin is the name of the remote repository that you want to pull from and branch is the name of the branch that you want to pull.

Conclusion

In this article, we have learned about Github and how to use it. We have also learned about how to setup an SSH key and add it to your Github account. We have also learned about how to push code to the remote repository.

Start your journey with ChaiDocs

All of our projects are available on [ChaiDocs](#). Feel free to check them out.